

Lec 10a

Application layer: DNS

Computer Networks (CSE 232)

Rinku Shah

These slides are inspired and adapted from:

1. *Computer Networking: A Top-Down Approach*
6th edition, Jim Kurose, Keith Ross, Pearson, 2017



INDRAPRASTHA INSTITUTE of
INFORMATION TECHNOLOGY
DELHI

DNS (Domain Name System)

- People are good at remembering **names** rather than **numbers**
- Communication with Internet hosts, routers, ...
 - Human use *names*
 - www.iiitd.ac.in
 - Network requires IP addresses (*numbers*)
 - IPv4: 32 bits
 - IPv6: 128 bits

DNS

- **Distributed** database
 - Hierarchy of many name servers
- Core Internet function
 - Implemented as application layer protocol
- DNS servers & hosts communicate to resolve names
 - address/name translation
- Complexity at the network edge
- Runs on top of **UDP**

How to map server names to IP addresses, and vice versa?

DNS structure & services

Q: Why not centralize DNS?

- single point of failure
- traffic volume
- distant centralized database
- maintenance

A: doesn't scale!

- Comcast DNS servers alone
 - 600B DNS queries/day
- Akamai DNS servers alone
 - 2.2T DNS queries/day

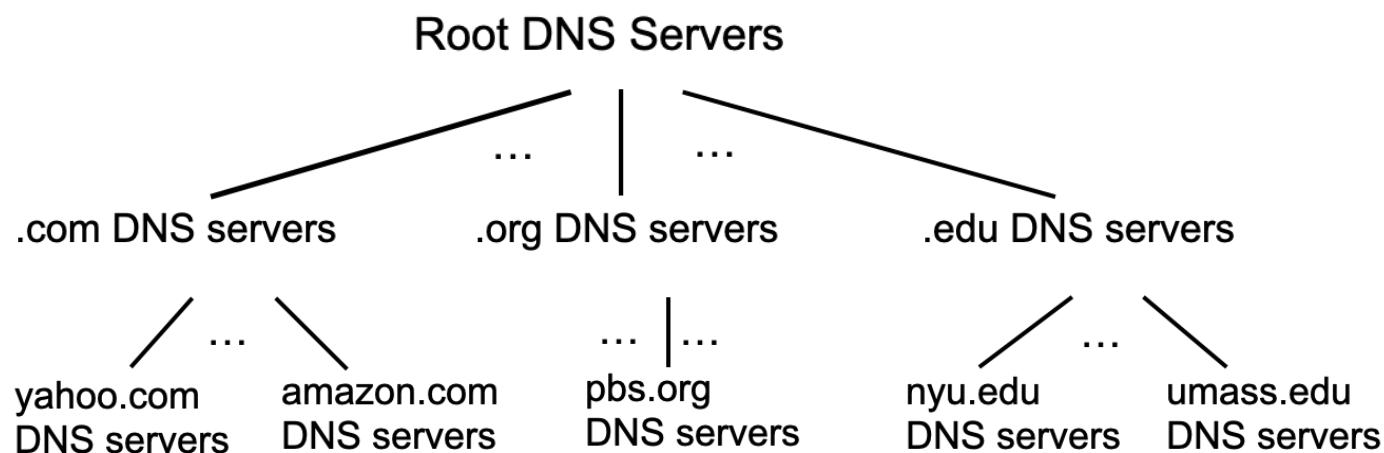
DNS services

- hostname-to-IP-address translation
- host aliasing
 - canonical, alias names
- mail server aliasing
- load distribution
 - replicated Web servers
 - many IP addresses correspond to one name

Requirements to be satisfied by DNS

- humongous distributed database
 - ~ billion records, each simple
- handles many *trillions* of queries/day
 - *many* more reads than writes
- *performance matters*
 - almost every Internet transaction interacts with DNS - msec count!
- organizationally, physically decentralized
 - millions of different organizations responsible for their records
- should be reliable & secure

DNS: a distributed, hierarchical database



Root

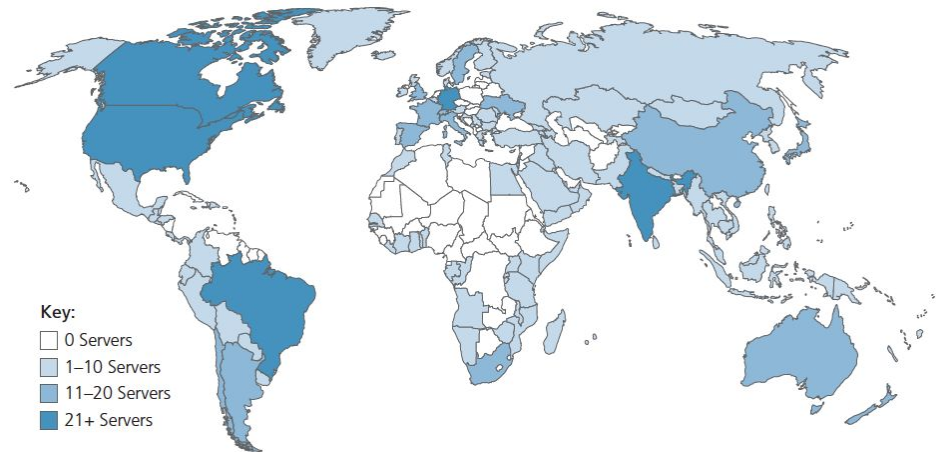
Top Level Domain

Authoritative

DNS: root name servers

- Root name server
 - contact-of-last-resort by name servers that can not resolve name
- *incredibly important*
 - Internet couldn't function without it!
- DNSSEC
 - Provides authentication, message integrity
- ICANN (Internet Corporation for Assigned Names and Numbers) manages root DNS domain

13 logical root name “servers”
worldwide each “server” replicated
many times (~200 servers in US)



Why only 13 root name servers?

<https://securitytrails.com/blog/dns-root-servers>

Root name servers in India

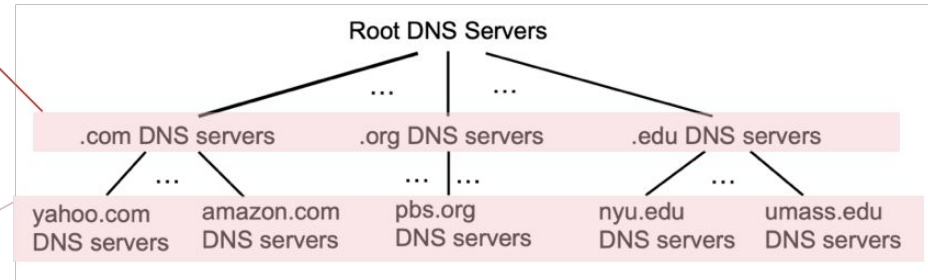
Currently, there is NO root server in India, but there are NINE instances

- three J-root instances — one in Delhi, one in Mumbai and one in Gorakhpur
- two L-root instances — one in Mumbai and one in Kolkata.
- The National Internet Exchange of India (NIXI) has sponsored three root server instances, one I-root in Mumbai, one K-root at Delhi and an F-Root in Chennai
- one D-root instance in Mumbai.

DNS: Top-Level Domain (TLD) servers & Authoritative servers

Top-Level Domain (TLD) servers

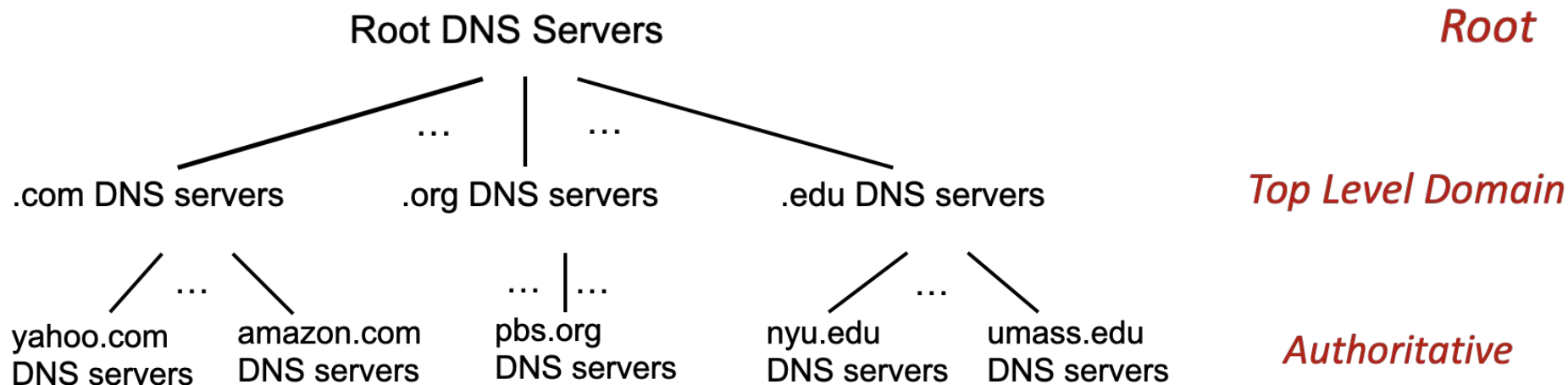
- responsible for .com, .org, .net, .edu, .aero, .jobs, .museums, and all top-level country domains, e.g., .in, .us, .cn, .uk, .fr, .ca, .jp



authoritative DNS servers

- organization's own DNS server(s)
 - providing authoritative hostname to IP mappings for organization's named hosts
- can be maintained by organization or service provider or third party
- usually more than one authoritative servers for REDUNDANCY

How is a DNS query resolved: 1st approximation



Client wants IP address for www.amazon.com; 1st approximation:

- client queries root server to find [.com](#) DNS server
- client queries [.com](#) DNS server to get [amazon.com](#) DNS server's IP address
- client queries [amazon.com](#) DNS server to get IP address for www.amazon.com

Local DNS name servers

- When host makes DNS query, it is sent to its *local* DNS server
 - Local DNS server uses **CACHING**, returns reply
 - from its local cache (**HIT**); name-to-address translation pairs (*possibly out of date!*)
 - **MISS**: forwards request into DNS hierarchy for resolution
 - Each ISP has local DNS name server; to find yours:
 - *Linux*
 - `$ cat /etc/resolv.conf`
 - *MacOS*
 - `% scutil --dns`
 - *Windows*
 - `> ipconfig /all`
- *local DNS server doesn't strictly belong to hierarchy*

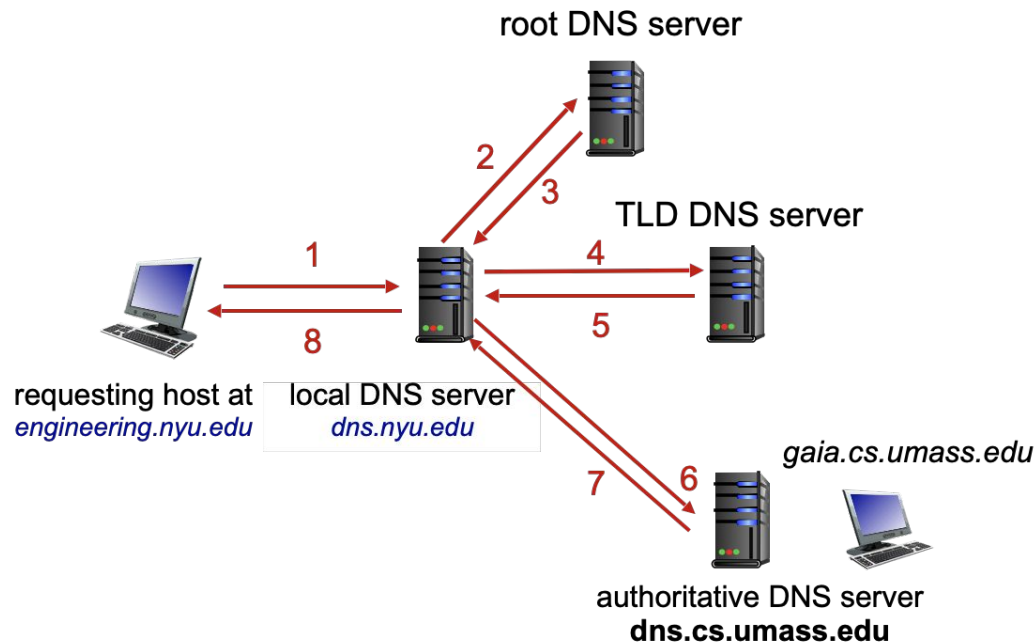
How does DNS server resolve DNS queries?

DNS name resolution: Iterated query

Example: host at *engineering.nyu.edu* wants IP address for *gaia.cs.umass.edu*

Iterated query:

- contacted server replies with name of server to contact
- “I don’t know this name, but ask this server”

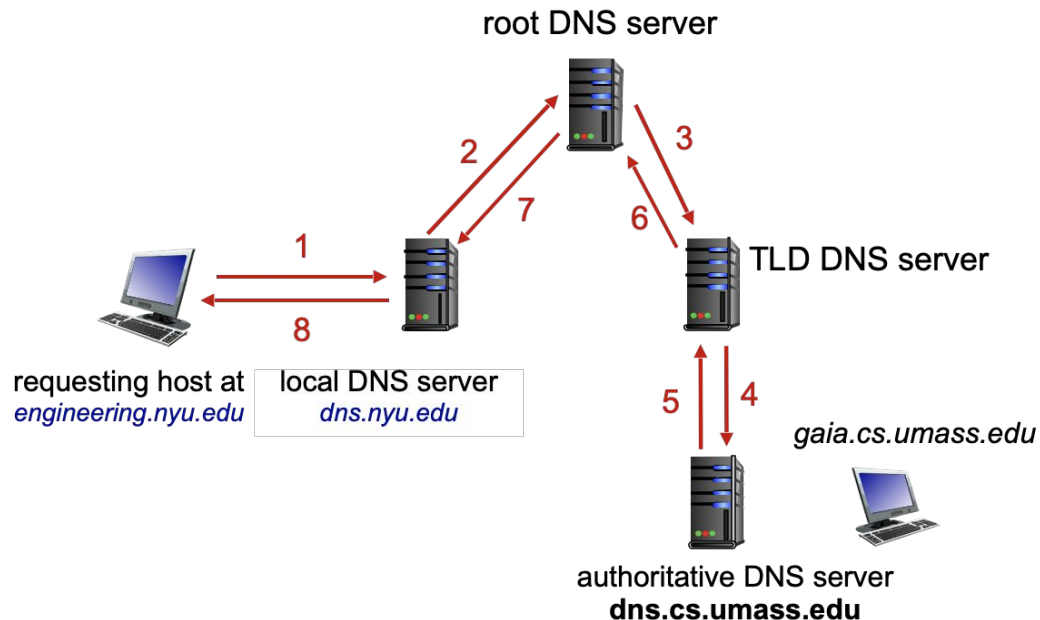


DNS name resolution: Recursive query

Example: host at *engineering.nyu.edu* wants IP address for *gaia.cs.umass.edu*

Recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?



Iterative query vs. Recursive query

Iterative DNS query	Recursive DNS query
Load at the local DNS server	Heavy load at upper levels of hierarchy of DNS system
Faster; this is due to more entries cached at local DNS server	Slower
More lines of code; complex tree data structure access.; relatively difficult to understand.	They require less lines of code than iteration; easier to understand

Caching DNS information

- once (any) name server learns mapping, it *cache*s mapping, and *immediately* returns a cached mapping in response to a query
 - caching improves response time
 - cache entries timeout (disappear) after some time (TTL)
 - TLD servers typically cached in local name servers
- cached entries may be *out-of-date*
 - if named host changes IP address, may not be known Internet-wide until all TTLs expire!
 - *best-effort name-to-address translation!*

DNS: distributed database storing resource records (RR)

RR format: (name, value, type, ttl)

type=A

- name is hostname
- value is IP address

type=NS

- name is domain (e.g., foo.com)
- value is hostname of authoritative name server for this domain

type=CNAME

- name is alias name for some “canonical” (the real) name
- www.ibm.com is really servereast.backup2.ibm.com
- value is canonical name

type=MX

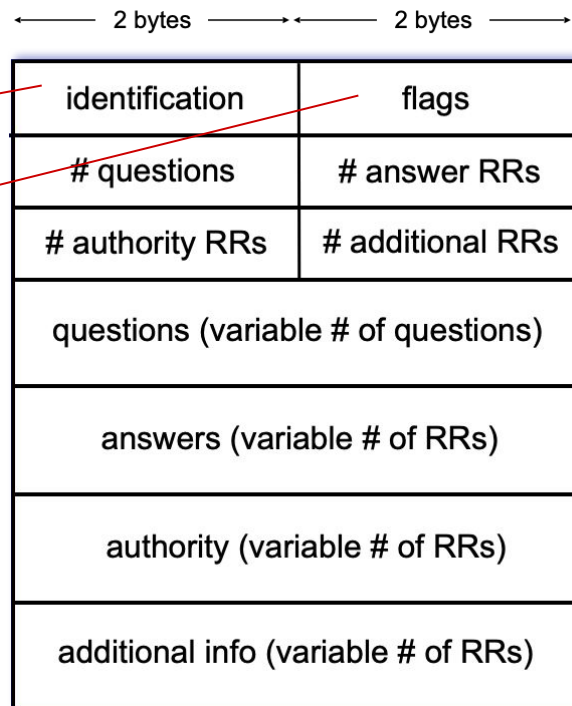
- value is name of SMTP mail server associated with name

DNS protocol messages

DNS *query* and *reply* messages, both have same *format*

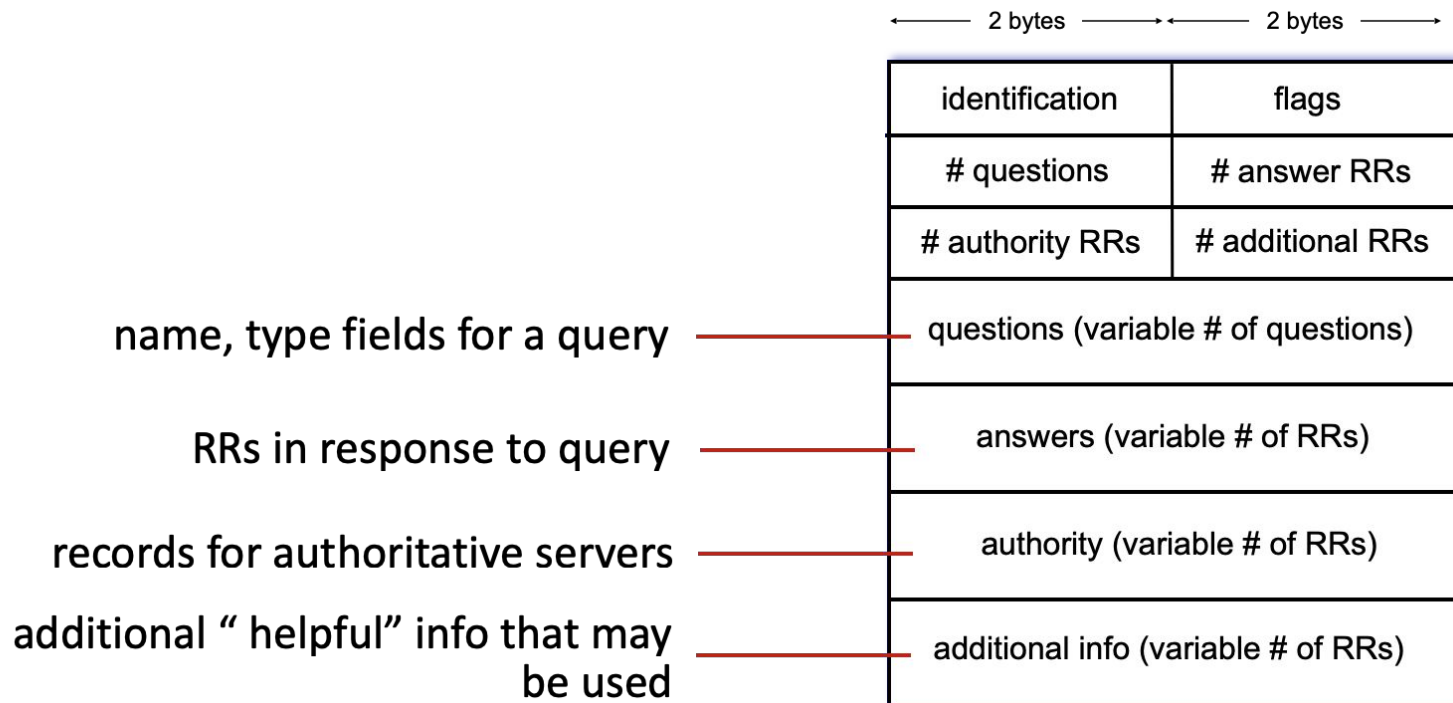
message header:

- **identification:** 16 bit # for query, reply to query uses same #
- **flags:**
 - query or reply
 - recursion desired
 - recursion available
 - reply is authoritative



DNS protocol messages

DNS *query* and *reply* messages, both have same *format*



Getting your info into the DNS

Example: We want to register *iiitd.ac.in* domain

1. Register name *iiitd.ac.in* at DNS registrar (e.g., GoDaddy India, BigRock India, ZNetLive)
2. Provide names, IP addresses of authoritative name server (primary and secondary)
3. Registrar inserts NS, A RRs into *.in* TLD server:
 - (*iiitd.ac.in*, *dns1.iiitd.ac.in*, NS)
 - (*dns1.iiitd.ac.in*, 103.25.231.30, A)
4. Create authoritative server locally with IP address 103.25.231.30
 - type A record for *www.iiitd.ac.in*
 - type MX record for *iiitd.ac.in*

DNS security (NOT PART OF THE COURSE)

DDoS attacks

- bombard root servers with traffic
 - not successful to date
 - traffic filtering
 - local DNS servers cache IPs of TLD servers, allowing root server bypass
- bombard TLD servers
 - potentially more dangerous

Spoofing attacks

- intercept DNS queries, returning bogus replies
 - DNS cache poisoning
 - RFC 4033: DNSSEC authentication services